

# TP3 — Administration & Automatisation Azure

Azure CLI · Bash · Python · Monitor · FinOps — Cas fil rouge ShopEasy

---

|           |                                          |
|-----------|------------------------------------------|
| Auteurs   | Olivier Falahi & Paul Claverie           |
| Formation | Mastère Dev Manager Full Stack — EFREI   |
| Module    | Optimisation du SI par l'apport du Cloud |
| Outils    | Azure CLI · Bash · Python (SDK Azure)    |
| Cloud     | Microsoft Azure (swedencentral)          |
| Année     | 2025 / 2026                              |

Scripts d'exploitation, inventaires, supervision, analyses FinOps & sécurité, rapport d'exploitation et procédures d'exécution.

# Sommaire et conformité au rendu

---

Ce document unique regroupe l'ensemble du rendu du TP3. Il répond aux **livrables attendus** (section 17 du sujet), produits sur l'environnement ShopEasy réellement déployé (Resource Group `rg-shopeasy-dev`, région `swedencentral`, souscription Azure for Students).

| Livrable attendu                                         | Format                        | Traité dans                                |
|----------------------------------------------------------|-------------------------------|--------------------------------------------|
| Variables utilisées                                      | <code>variables.sh</code>     | Annexe B                                   |
| Scripts Bash (inventaire, gestion VM, contrôle de santé) | <code>.sh</code>              | Annexe B + Compte rendu (ateliers 5, 6, 9) |
| Script Python (inventaire via SDK)                       | <code>.py</code>              | Annexe B + Compte rendu (atelier 10)       |
| Exports (JSON, TSV, TXT, CSV)                            | dossier <code>exports/</code> | ZIP de rendu                               |
| Captures (preuves d'exécution)                           | PNG                           | Annexe C                                   |
| Rapport d'exploitation                                   | Markdown / PDF                | Rapport d'exploitation                     |
| Quiz final                                               | texte                         | Quiz (20 réponses)                         |

## Contenu

---

1. Note technique (synthèse des choix + architecture d'exploitation)
2. Compte rendu des ateliers (1 à 12)
3. Analyse FinOps & sécurité (atelier 11 + corrigé sécurité)
4. Rapport d'exploitation ShopEasy (10 sections + tableau de synthèse)
5. Quiz de validation (20 questions)
6. Annexe A — Arborescence du projet `tp3/`
7. Annexe B — Code source (variables, scripts Bash, script Python)
8. Annexe C — Preuves d'exécution (captures)

# Note technique — Exploitation de ShopEasy sur Azure (TP3)

Synthèse à destination d'une équipe CloudOps reprenant l'administration de l'environnement.

## Objet

Cette note résume le passage d'une logique de **déploiement** (TP1/TP2) à une logique d'**exploitation**. L'infrastructure ShopEasy (déployée par Terraform) est désormais administrée, inventoriée, supervisée et optimisée par des commandes reproductibles et des scripts relançables, sans dépendre du portail Azure.

## Choix techniques

- **Azure CLI** comme outil central d'administration : commandes scriptables, sorties JSON/table/TSV exploitables.
- **Scripts Bash** ( `inventory.sh` , `vm-power.sh` , `healthcheck.sh` ) pour les tâches récurrentes, avec `set -euo pipefail` , gestion d'erreur, confirmation des actions destructives et journalisation.
- **Python + SDK Azure** ( `inventory.py` ) pour produire un inventaire structuré et un CSV exploitable en tableur.
- **Azure Monitor** pour la supervision (métrique CPU + alerte seuil).
- **Tags d'exploitation** appliqués en mode `Merge` pour préserver l'état Terraform.
- **Sécurité** : authentification Entra ID pour le stockage (pas de clés), accès public désactivé, SSH restreint, garde-fou anti-production dans les scripts.

## Architecture d'exploitation

```
graph LR
    Ops["Equipe CloudOps"] --> CLI["Azure CLI"]
    Ops --> Scripts["Scripts Bash / Python"]
    CLI --> ARM["Azure Resource Manager"]
    Scripts --> ARM
    ARM --> RG["rg-shopeasy-dev"]
    RG --> VMs["2 VM web (Nginx)"]
    RG --> LB["Load Balancer"]
    RG --> ST["Storage (conteneur operations)"]
    RG --> NSG["NSG web + data"]
    Monitor["Azure Monitor"] --> VMs
    Monitor --> Alert["Alerte CPU > 80%"]
    Scripts --> ST
```

Schéma d'architecture (notation Mermaid).

## Adaptation à l'environnement réel

| Variable sujet | Valeur réelle                                                      |
|----------------|--------------------------------------------------------------------|
| LOCATION       | swedencentral (policy Azure for Students, francecentral interdite) |
| VM1 / VM2      | vm-shopeasy-dev-web-1 / vm-shopeasy-dev-web-2                      |
| STORAGE        | shopeasydevdocsa0rnay (suffixe aléatoire Terraform)                |
| CONTAINER      | operations (créé au TP3)                                           |

## Points de vigilance

- `az vm stop` ne stoppe pas la facturation compute : utiliser `deallocate`.
- `az vm update --set tags` échoue sur cette souscription (« zone movement ») : préférer `az tag update --operation Merge`.
- L'auth `--auth-mode login` du stockage exige un rôle data-plane RBAC.
- Les commandes sont rejouables et les scripts sont versionnés : aucune action n'est « one-shot » non documentée.

## Conclusion

L'environnement est exploitable par un tiers à partir du mini-kit livré (scripts + variables + rapport). Les écarts restants (tags incomplets, budget à définir) sont identifiés et priorisés dans le rapport d'exploitation.

# TP3 — Administration Cloud et Automatisation Azure — Compte rendu des ateliers

---

Cas fil rouge : exploitation de l'environnement **ShopEasy** sur Azure, déployé au TP2 avec Terraform. Resource Group : `rg-shopeasy-dev` — Région : `swedencentral` — Souscription : Azure for Students.

**Note de contexte.** Le sujet propose des noms génériques ( `vm-shopeasy-web-01` , `francecentral` , `stshopeasydev001` ). Ils sont **adaptés** aux ressources réellement déployées par Terraform au TP2 et conservés dans [variables.sh](#) . La région `francecentral` étant interdite par la policy Azure for Students, l'environnement vit à `swedencentral` .

**État de l'environnement.** Infrastructure **redéployée et validée le 26/06/2026** ( `terraform apply` → 24 ressources). Valeurs réelles : Load Balancer `20.91.233.129` , VM web-1 `135.225.37.122` , VM web-2 `4.223.122.111` , Storage `shopeasydevdocsa0rnay` . Toutes les commandes ci-dessous ont été exécutées réellement sur cet environnement ; les preuves sont regroupées en **Annexe C** du PDF et listées dans [06\\_captures\\_a\\_faire.md](#) .

## Variables utilisées

---

```
export RG="rg-shopeasy-dev"
export LOCATION="swedencentral"
export VM1="vm-shopeasy-dev-web-1"
export VM2="vm-shopeasy-dev-web-2"
export STORAGE="shopeasydevdocsa0rnay"
export CONTAINER="operations"
export AZURE_SUBSCRIPTION_ID="cdca6d99-645a-4251-85e1-f078d1bd66ff"
```

Le fichier complet et commenté est livré dans [tp3/variables.sh](#) et se charge via `source tp3/variables.sh` .

---

## Atelier 1 — Prise en main d'Azure CLI

---

**Objectif.** Vérifier l'accès Azure, identifier la souscription active et se positionner sur le bon groupe de ressources.

Commandes exécutées :

```
az account show --output table
az configure --defaults group=$RG location=$LOCATION
az configure --list-defaults --output table
az group show --name $RG --query "{Nom:name,Region:location,Etat:properties.provisioningState}" --
output table
```

Résultats observés :

| EnvironmentName | IsDefault | Name               | State   | TenantDefaultDomain |
|-----------------|-----------|--------------------|---------|---------------------|
| AzureCloud      | True      | Azure for Students | Enabled | efrei.net           |

| Name     | Source                   | Value           |
|----------|--------------------------|-----------------|
| group    | /home/paul/.azure/config | rg-shopeasy-dev |
| location | /home/paul/.azure/config | swedencentral   |

| Nom             | Region        | Etat      |
|-----------------|---------------|-----------|
| rg-shopeasy-dev | swedencentral | Succeeded |

**Analyse.** La souscription active est bien le compte de formation (Azure for Students, tenant `efrei.net`) — vérifié au préalable par le garde-fou `scripts/azure-account.sh guard` qui interdit tout déploiement sur un abonnement d'entreprise. La configuration de valeurs par défaut ( `group` , `location` ) évite de répéter `--resource-group` et `--location` sur chaque commande, ce qui réduit le risque d'erreur de saisie en exploitation.

**Livrable.** Capture `atelier_01-account-defaults.png` .

## Atelier 2 — Inventorier les ressources Azure

**Objectif.** Produire un inventaire lisible et reproductible des ressources ShopEasy, sans passer par le portail.

Commandes exécutées :

```
az resource list --resource-group $RG --query "[].{Nom:name,Type:type,Region:location}" --output
table
az resource list --resource-group $RG --output json > tp3/exports/resources.json
az resource list --resource-group $RG --query "[].{Nom:name,Type:type,Region:location}" --output tsv
> tp3/exports/resources.tsv
az resource list --resource-group $RG --query "[].type" --output tsv | sort | uniq -c
```

Décompte par type de ressource (14 ressources au total) :

| Nb | Type Azure              |
|----|-------------------------|
| 2  | Microsoft.Compute/disks |

| Nb | Type Azure                              |
|----|-----------------------------------------|
| 2  | Microsoft.Compute/virtualMachines       |
| 1  | Microsoft.Network/loadBalancers         |
| 2  | Microsoft.Network/networkInterfaces     |
| 2  | Microsoft.Network/networkSecurityGroups |
| 3  | Microsoft.Network/publicIPAddresses     |
| 1  | Microsoft.Network/virtualNetworks       |
| 1  | Microsoft.Storage/storageAccounts       |

## Tableau d'inventaire commenté

| Nom ressource              | Type Azure            | Région        | Rôle dans l'architecture                           |
|----------------------------|-----------------------|---------------|----------------------------------------------------|
| vnet-shopeasy-dev          | virtualNetworks       | swedencentral | Réseau privé<br>10.20.0.0/16 (subnets web + data)  |
| nsg-shopeasy-dev-web       | networkSecurityGroups | swedencentral | Pare-feu du subnet web (HTTP 80 + SSH restreint)   |
| nsg-shopeasy-dev-data      | networkSecurityGroups | swedencentral | Pare-feu du subnet data (SQL depuis web, Deny-All) |
| lb-shopeasy-dev-web        | loadBalancers         | swedencentral | Répartiteur HTTP public (round-robin sur les 2 VM) |
| pip-shopeasy-dev-lb        | publicIPAddresses     | swedencentral | IP publique du Load Balancer ( 20.91.233.129 )     |
| pip-shopeasy-dev-web-1/2   | publicIPAddresses     | swedencentral | IP publiques directes des VM (administration)      |
| nic-shopeasy-dev-web-1/2   | networkInterfaces     | swedencentral | Cartes réseau des VM (rattachées au backend pool)  |
| vm-shopeasy-dev-web-1/2    | virtualMachines       | swedencentral | Serveurs web Nginx ( Standard_B2ts_v2 )            |
| vm-...-web-1/2_0s-Disk_... | disks                 | swedencentral | Disques OS managés des VM                          |
| shopeasydevdocsa0r-nay     | storageAccounts       | swedencentral | Compte de stockage des documents/rapports          |

**Analyse.** L'inventaire est rejouable : les fichiers [exports/resources.json](#) (392 lignes, machine-exploitable) et [exports/resources.tsv](#) (réutilisable dans un script Bash via `cut / awk` ) constituent une photographie datée du périmètre. Le décompte par type permet de détecter immédiatement une dérive (par exemple un disque orphelin ou une IP publique inattendue).

**Livrible.** `exports/resources.json` , `exports/resources.tsv` , capture `atelier_02-resource-list.png` .

## Atelier 3 — Normaliser les tags d'exploitation

**Objectif.** Appliquer une stratégie de tags minimale pour faciliter l'exploitation, le suivi des coûts (FinOps) et l'identification des responsabilités.

Commandes exécutées (mode **Merge** pour ne pas écraser les tags déjà posés par Terraform) :

```
RG_ID=$(az group show --name $RG --query id -o tsv)
az tag update --resource-id "$RG_ID" --operation Merge \
  --tags Application=shopeasy Environment=dev Owner=formation CostCenter=cloud-lab ManagedBy=cli

VM1_ID=$(az vm show -g $RG -n $VM1 --query id -o tsv)
az tag update --resource-id "$VM1_ID" --operation Merge \
  --tags Application=shopeasy Environment=dev Role=web Owner=formation
```

**Point d'attention.** La commande `az vm update --set tags...` du sujet renvoie sur cette souscription l'erreur `Operation 'Enabling zone movement' is not supported` . La pose de tags via `az tag update --operation Merge` est plus robuste et **préserve** les tags Terraform ( `project` , `environment` , `owner` , `managed_by` , `cost_center` ).

Tags du groupe de ressources après application :

```
{
  "Application": "shopeasy", "CostCenter": "cloud-lab", "ManagedBy": "cli",
  "cost_center": "cloud-training", "environment": "dev",
  "managed_by": "terraform", "owner": "formation", "project": "shopeasy"
}
```

Tags d'une VM après application :

```
{
  "Application": "shopeasy", "Role": "web",
  "cost_center": "cloud-training", "environment": "dev",
  "managed_by": "terraform", "owner": "formation", "project": "shopeasy"
}
```

**Pourquoi les tags sont utiles (FinOps et gouvernance).** Les tags transforment un parc de ressources opaque en un parc interrogeable : la facturation Azure Cost Management peut être ventilée par `Application` ou `CostCenter` pour répondre à « combien coûte ShopEasy ce mois-ci ? » ; le tag `Owner` désigne un responsable joignable en cas d'incident ; `Environment` permet de cibler les actions d'arrêt nocturne sur les seules ressources `dev` ; `ManagedBy` distingue ce qui est piloté par l'IaC de ce qui a été modifié à la



main (signal de dérive). Sans tags, ces questions imposent un inventaire manuel coûteux et faillible.

**Livrable.** Captures atelier\_03-tags.png (sortie CLI des tags RG + VM).

## Atelier 4 — Administrer les machines virtuelles

**Objectif.** Réaliser les opérations courantes sur les VM : lister, vérifier l'état, redémarrer, désallouer et exécuter une commande distante.

État synthétique des VM :

```
az vm list --resource-group $RG --show-details \
  --query "[].{Nom:name,Etat:powerState,IP:publicIps,Taille:hardwareProfile.vmSize}" \
  --output table
```

| Nom                   | Etat       | IP             | Taille           |
|-----------------------|------------|----------------|------------------|
| vm-shopeasy-dev-web-1 | VM running | 135.225.37.122 | Standard_B2ts_v2 |
| vm-shopeasy-dev-web-2 | VM running | 4.223.122.111  | Standard_B2ts_v2 |

Commande distante (vérification du serveur web sans SSH interactif) :

```
az vm run-command invoke --resource-group $RG --name $VM1 --command-id RunShellScript \
  --scripts "hostname && uptime && systemctl is-active nginx && curl -s localhost | grep -oiE
'ShopEasy[^\<]*'"
```

```
vm-shopeasy-dev-web-1
08:09:56 up 4 min, 0 users, load average: 0.10, 0.15, 0.08
active
ShopEasy - serveur web 1
```

Redémarrage puis vérification de l'état :

```
az vm restart --resource-group $RG --name $VM1
az vm get-instance-view -g $RG -n $VM1 \
  --query "instanceView.statuses[?starts_with(code,'PowerState/')].displayStatus" -o tsv
# -> VM running
```

### Tableau des actions VM

| VM                    | Taille           | IP publique    | État       | Action réalisée                  | Résultat                                            |
|-----------------------|------------------|----------------|------------|----------------------------------|-----------------------------------------------------|
| vm-shopeasy-dev-web-1 | Standard_B2ts_v2 | 135.225.37.122 | VM running | run-command (nginx) puis restart | Nginx actif, page « serveur web 1 », redémarrage OK |

| VM                    | Taille           | IP publique   | État       | Action réalisée                                      | Résultat                       |
|-----------------------|------------------|---------------|------------|------------------------------------------------------|--------------------------------|
| vm-shopeasy-dev-web-2 | Standard_B2ts_v2 | 4.223.122.111 | VM running | deallocate puis start (via vm-powershell, atelier 6) | Cycle d'arrêt/démarrage valide |

**Point de vigilance — stop vs deallocate.** `az vm stop` éteint l'OS mais **continue de facturer le compute** (la VM reste allouée sur un hôte). Pour cesser la facturation compute, il faut `az vm deallocate` (état Stopped (deallocated)). C'est le geste FinOps de référence pour un environnement de développement inactif.

**Livrable.** Tableau ci-dessus + captures `atelier_04-vm-runcommand.png`.

## Atelier 5 — Script Bash d'inventaire ( `inventory.sh` )

**Objectif.** Transformer les commandes manuelles d'inventaire en script réutilisable et enrichi de compteurs.

Le script livré `scripts/inventory.sh` reprend la base du sujet et ajoute les améliorations demandées : nombre total de ressources, nombre de VM, nombre de comptes de stockage, nombre de ressources sans tag `Application`, et un avertissement listant les VM encore en cours d'exécution (coût compute facturé). Les exports sont horodatés dans `exports/`.

Exécution réelle :

```
Inventaire Azure - groupe de ressources : rg-shopeasy-dev
Date : 20260626-101400
-----
Export des ressources (JSON)...
Export des VM (table)...
-----
Synthese :
  Ressources totales      : 14
  Machines virtuelles     : 2
  Comptes de stockage     : 1
  Ressources sans tag App. : 12
-----
ATTENTION : VM en cours d'execution (cout compute facture) :
  - vm-shopeasy-dev-web-1
  - vm-shopeasy-dev-web-2
-----
Export termine dans exports/ :
```

- resources-20260626-101400.json
- vms-20260626-101400.txt

**Analyse.** Le script est robuste ( `set -euo pipefail` ), vérifie l'existence du groupe avant d'agir, et accepte le RG en argument ( `./inventory.sh rg-shopeasy-dev` ) avec une valeur par défaut. Le compteur « 12 ressources sans tag `Application` » est un signal d'exploitation réel : seuls le groupe et les deux VM ont reçu le tag `Application` à l'atelier 3 ; les ressources réseau et le stockage restent à normaliser (recommandation FinOps).

**Livable.** Script `inventory.sh` , capture `atelier_05-inventory-script.png` , fichiers `exports/resources-*.json` et `exports/vms-*.txt` .

## Atelier 6 — Automatiser l'arrêt/démarrage des VM ( `vm-power.sh` )

**Objectif.** Piloter l'alimentation des VM de développement avec un script sûr.

Le script `scripts/vm-power.sh` prend en charge `start` | `stop` | `deallocate` | `status` sur toutes les VM du groupe, avec les mesures de sécurité demandées : confirmation interactive avant `stop` / `deallocate` , refus d'exécution si le nom du groupe contient `prod` , et journalisation horodatée dans `logs/vm-power.log` .

Démonstrations exécutées :

```
# 1. Garde-fou anti-production (sortie immédiate, code 2)
$ ./scripts/vm-power.sh rg-shopeasy-prod deallocate
Refus : 'rg-shopeasy-prod' ressemble a un groupe de PRODUCTION. Action interdite par ce script.

# 2. Confirmation refusée (réponse "non") -> aucune action
$ printf 'non\n' | ./scripts/vm-power.sh rg-shopeasy-dev deallocate
Action 'deallocate' annulee par l'utilisateur

# 3. Cycle réel deallocate -> status -> start
$ printf 'oui\n' | ./scripts/vm-power.sh rg-shopeasy-dev deallocate
VM desallouee (cout compute stoppe) : vm-shopeasy-dev-web-1 / -web-2
$ ./scripts/vm-power.sh rg-shopeasy-dev status
Etat : vm-shopeasy-dev-web-1 -> VM deallocated
$ ./scripts/vm-power.sh rg-shopeasy-dev start
VM demarree : vm-shopeasy-dev-web-1 / -web-2
```

Extrait du journal `logs/vm-power.log` :

```
2026-06-26 10:19:54 | rg-shopeasy-dev | deallocate | VM desallouee (cout compute stoppe) : vm-shopeasy-dev-web-1
2026-06-26 10:20:37 | rg-shopeasy-dev | status      | Etat : vm-shopeasy-dev-web-2 -> VM deallocated
2026-06-26 10:21:23 | rg-shopeasy-dev | start      | VM demarree : vm-shopeasy-dev-web-1
```

**Mesures de sécurité ajoutées.** (1) Le filtre `*prod*` empêche un usage accidentel du script sur un environnement de production ; (2) la confirmation `oui/non` protège contre les

arrêts involontaires ; (3) le journal trace qui a fait quoi et quand, ce qui est indispensable pour l'audit d'exploitation. `deallocate` est privilégié à `stop` car seul `deallocate` libère le compute facturé.

**Livvable.** Script `vm-power.sh`, journal `logs/vm-power.log`, capture `atelier_06-vm-power.png`.

## Atelier 7 — Exploiter un Storage Account

**Objectif.** Disposer d'un espace de stockage privé pour déposer les rapports d'exploitation.

Le compte de stockage `shopeasydevdocs00rnay` (créé au TP2) est réutilisé. Constat initial : l'accès public au niveau blob était **autorisé** (`allowBlobPublicAccess: True`), car le `storage.tf` du TP2 ne le restreint pas explicitement. C'est précisément un écart d'exploitation que le TP3 corrige.

Commandes exécutées :

```
# Durcissement : interdire l'accès public au niveau du compte
az storage account update --name $STORAGE --resource-group $RG --allow-blob-public-access false

# Conteneur privé d'exploitation
az storage container create --account-name $STORAGE --name operations --auth-mode login --public-access off

# Dépôt des exports (auth Entra ID, sans clé de compte)
az storage blob upload --account-name $STORAGE --container-name operations \
  --file exports/resources.tsv --name inventaire/resources.tsv --auth-mode login --overwrite true
```

**Point RBAC.** Le téléversement via `--auth-mode login` exige un rôle data-plane. Le rôle **Storage Blob Data Contributor** a été attribué à l'utilisateur sur le scope du compte de stockage (`az role assignment create`), ce qui évite d'exposer les clés de compte — bonne pratique de sécurité d'exploitation.

Blobs déposés dans `operations` :

| Nom                                       | Taille |
|-------------------------------------------|--------|
| inventaire/resources-20260626-101400.json | 6793   |
| inventaire/resources.tsv                  | 1059   |
| rapports/rapport-test.txt                 | 62     |

**Pourquoi les rapports d'exploitation ne doivent pas être publics.** Un rapport d'exploitation décrit la topologie (noms de VM, IP, types de ressources), l'état de santé et parfois les failles connues. Exposé publiquement, il constitue une cartographie offerte à un attaquant. L'accès doit donc passer par l'authentification Entra ID et le RBAC, jamais par un blob anonyme.

**Livrable.** Capture atelier\_07-storage-blobs.png (liste des blobs + allowBlobPublicAccess: False ).

## Atelier 8 — Surveiller les métriques avec Azure Monitor

**Objectif.** Lire les métriques d'une VM et créer une alerte sur une ressource critique.

Trois métriques (sur cinq identifiées) pertinentes pour un serveur web :

| Métrique               | Unité   | Usage                    |
|------------------------|---------|--------------------------|
| Percentage CPU         | Percent | Saturation processeur    |
| Available Memory Bytes | Bytes   | Pression mémoire         |
| Network In Total       | Bytes   | Volume de trafic entrant |

Lecture de la métrique CPU (5 derniers points, intervalle 1 min) :

| Heure                | CPU_moyen |
|----------------------|-----------|
| 2026-06-26T08:13:00Z | 0.87      |
| 2026-06-26T08:15:00Z | 0.125     |
| 2026-06-26T08:17:00Z | 0.125     |

Création de l'alerte CPU :

```
az monitor metrics alert create \  
  --name "alert-cpu-high-$VM1" --resource-group $RG --scopes "$VM_ID" \  
  --condition "avg Percentage CPU > 80" --evaluation-frequency 1m --window-size 5m --severity 3
```

| Nom                                  | Severite | Active | Fenetre |
|--------------------------------------|----------|--------|---------|
| alert-cpu-high-vm-shopeasy-dev-web-1 | 3        | True   | PT5M    |

**Justification du seuil (80 % sur 5 min).** Un seuil à 80 % moyenné sur une fenêtre de 5 minutes évite les faux positifs liés aux pics ponctuels (démarrage de service, apt , cron) tout en détectant une saturation durable avant la dégradation du service. Une fenêtre plus courte rendrait l'alerte bruyante ; un seuil plus haut (95 %) déclencherait trop tard.

**Deux alertes complémentaires proposées pour ShopEasy.** (1) Disponibilité HTTP du Load Balancer (sonde d'intégrité Health Probe Status < 100 %) pour détecter une VM sortie du backend ; (2) Available Memory Bytes sous un seuil critique (par ex. < 200 Mo) pour anticiper un OOM. **Limite d'une alerte trop sensible :** elle génère de la fatigue d'alerte (les équipes finissent par l'ignorer) ; **trop large :** elle masque la cause réelle et retarde le diagnostic.

**Livrable.** Capture atelier\_08-monitor-alert.png (métrique CPU + alerte créée).

## Atelier 9 — Script de contrôle de santé ( `healthcheck.sh` )

**Objectif.** Vérifier rapidement que l'environnement est dans un état acceptable.

Le script `scripts/healthcheck.sh` enchaîne huit contrôles : existence du groupe, présence d'au moins une VM (avec état), ressources sans tag `Application`, tag `Owner` sur le groupe, alertes Azure Monitor, existence du compte de stockage, existence du conteneur `operations`, et présence d'au moins une règle NSG autorisant HTTP/HTTPS. Il renvoie un code de sortie non nul si un avertissement est détecté (utilisable dans une chaîne CI ou un cron).

Exécution réelle :

```
1. Verification du groupe de ressources      OK - groupe trouve
2. Verification des VM                      OK - 2 VM presente(s)
3. Verification des ressources sans tag App.  WARN- 12 ressource(s) sans tag Application
4. Verification du tag Owner sur le groupe   OK - Owner = formation
5. Verification des alertes Azure Monitor    OK - 1 alerte
6. Verification du compte de stockage        OK - shopeasydevdocsa0rnay
7. Verification du conteneur 'operations'    OK - present
8. Regle NSG autorisant HTTP/HTTPS          OK - 1 regle
Resultat : avertissement(s) detecte(s)
```

**Contrôles ajoutés (par rapport au modèle du sujet).** Storage Account, conteneur `operations`, compteur de VM, tag `Owner`, et règle NSG HTTP/HTTPS. Le seul `WARN` restant (tag `Application` manquant sur 12 ressources) reflète fidèlement l'état réel et invite à compléter la normalisation des tags — le script joue donc bien son rôle de détecteur d'écart.

**Livable.** Script `healthcheck.sh`, capture `atelier_09-healthcheck.png`.

## Atelier 10 — Automatiser avec Python et le SDK Azure ( `inventory.py` )

**Objectif.** Produire un inventaire programmable et un export CSV exploitable dans un tableur.

Le script `python/inventory.py` utilise `DefaultAzureCredential` (réutilise la session `az login`), liste les ressources et les VM, et écrit un CSV ( `exports/inventory.csv` ) avec les colonnes : nom, type, région, tags, rôle supposé. Le CSV est encodé en UTF-8 avec BOM et séparateur `;` pour une ouverture propre dans Excel/LibreOffice.

```
15 ressource(s) exportee(s) dans exports/inventory.csv
```

Extrait du CSV :

```
nom;type;region;tags;role
vnet-shopeasy-dev;Microsoft.Network/
```

```
virtualNetworks;swedencentral;"...environment=dev;owner=formation...";Réseau privé (VNet)
vm-shopeasy-dev-web-1;Microsoft.Compute/
virtualMachines;swedencentral;"...Application=shopeasy;Role=web...";Serveur web (VM Linux Nginx)
```

**Quand préférer Python à Bash ?** Bash excelle pour enchaîner des commandes az simples ; Python devient pertinent dès qu'il faut structurer des données (CSV, JSON imbriqué), gérer des erreurs finement, ou réutiliser la logique dans une application. Le SDK Azure offre des objets typés plutôt que du parsing de texte.

**Livable.** Script `inventory.py` , `requirements.txt` , fichier `exports/inventory.csv` , capture `atelier_10-python-csv.png` .

---

## Atelier 11 — Analyse FinOps d'exploitation

Traité en détail dans [03\\_analyse\\_finops\\_securite.md](#) : tableau des actions FinOps, stratégie d'arrêt/démarrage des VM de développement, politique de tags, seuil d'alerte budgétaire et trois recommandations pour la DSI.

---

## Atelier 12 — Rapport d'exploitation ShopEasy

Produit dans [04\\_rapport\\_exploitation.md](#) : rapport structuré en dix sections avec le tableau de synthèse (Inventaire, Tags, VM, Stockage, Monitoring, Sécurité, FinOps → Conforme / partiel / non conforme).

# TP3 — Analyse FinOps et sécurité d'exploitation (ateliers 11 et corrigé sécurité)

Périmètre : rg-shopeasy-dev (souscription Azure for Students, swedencentral ). Deux VM Standard\_B2ts\_v2 , un Load Balancer, un compte de stockage, deux NSG.

## 1. Analyse FinOps (atelier 11)

### Ressources génératrices de coût

Dans cet environnement de développement, le **compute** domine la facture : les deux VM Standard\_B2ts\_v2 tournent 24h/24 alors que l'usage réel est ponctuel. Viennent ensuite les **disques OS managés** (facturés même VM désallouée), les **IP publiques Standard** (facturées à l'heure), puis le **stockage** (faible volume ici).

### Tableau des actions FinOps

| Action                           | Effet attendu                               | Risque ou limite                | Décision pour ShopEasy dev                                                          |
|----------------------------------|---------------------------------------------|---------------------------------|-------------------------------------------------------------------------------------|
| Désallouer les VM hors usage     | Réduit le coût compute (le plus gros poste) | Indisponibilité pendant l'arrêt | <b>Retenue</b> — arrêt nocturne et week-end via <code>vm-power.sh deallocate</code> |
| Réduire la taille des VM         | Réduit le coût mensuel                      | Performance plus faible         | Possible — B2ts_v2 est déjà petit ; à réévaluer si CPU < 5 % en continu             |
| Supprimer les disques inutilisés | Réduit le stockage facturé                  | Risque de perte de données      | À faire après <code>terraform destroy</code> (pas de disque orphelin actuellement)  |
| Standardiser les tags            | Meilleure ventilation des coûts             | Discipline d'équipe nécessaire  | <b>Retenue</b> — compléter le tag Application sur les 12 ressources restantes       |
| Définir un budget                | Alerte en cas de dérive                     | Ne bloque pas la dépense        | <b>Retenue</b> — budget mensuel + alerte à 80 %                                     |

### Stratégie d'arrêt/démarrage des VM de développement

Les VM dev n'ont pas besoin d'être disponibles la nuit ni le week-end. La stratégie retenue : **désallocation automatique** en dehors des heures ouvrées (par exemple 20h-8h et le week-end), via le script `vm-power.sh` déclenché par un cron / une Azure Automation runbook. Sur une base 24/7 ramenée à ~50 h/semaine d'allocation,



l'économie compute approche **60-70 %**. Le démarrage matinal peut être planifié ou laissé à la demande ( `vm-power.sh start` ).

## Politique de tags FinOps

Tags obligatoires sur **toute** ressource : `Application` (ventilation par produit), `Environment` (cible des actions d'arrêt), `Owner` (responsable), `CostCenter` (refacturation), `ManagedBy` (IaC vs manuel). Un contrôle automatisé (le compteur « ressources sans tag `Application` » d' `inventory.sh` ) signale les manquements ; en production, une **Azure Policy** en mode `deny` ou `append` imposerait les tags à la création.

## Seuil d'alerte budgétaire

Pour un environnement de formation, un **budget mensuel de l'ordre de 30 € à 50 €** avec une **alerte à 80 %** (et une seconde à 100 %) est pertinent : assez bas pour réagir avant l'épuisement du crédit Azure for Students, assez haut pour ne pas alerter sur le bruit quotidien. Le budget se définit dans Cost Management ( `az consumption budget create` ).

## Trois recommandations FinOps pour la DSI

1. **Automatiser la désallocation nocturne** des environnements non-prod : c'est le levier d'économie le plus rapide et le plus sûr (réversible).
2. **Imposer les tags via Azure Policy** : sans tags fiables, l'analyse de coût par application est impossible et la refacturation interne échoue.
3. **Mettre en place un budget avec alertes** par environnement, couplé à une revue mensuelle des ressources orphelines (disques, IP publiques non attachées).

## 2. Analyse sécurité d'exploitation (corrigé sécurité)

### Risques identifiés et mesures correctives

| Risque                  | Constat sur l'environnement                                                                                         | Mesure corrective                                                                                       | Statut   |
|-------------------------|---------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|----------|
| SSH exposé à Internet   | NSG <code>Allow-SSH-Admin</code> limité à l'IP de l'administrateur ( <code>/32</code> ), pas <code>0.0.0.0/0</code> | Maintenir la restriction ; en prod, passer par Azure Bastion                                            | Conforme |
| Stockage public         | <code>allowBlobPublicAccess</code> était à <code>True</code> (défaut Terraform)                                     | Désactivé en TP3 ( <code>--allow-blob-public-access false</code> ) + conteneur privé                    | Corrigé  |
| Accès stockage par clés | Risque de fuite de clés de compte                                                                                   | Auth Entra ID ( <code>--auth-mode login</code> ) + rôle RBAC <code>Storage Blob Data Contributor</code> | Conforme |

| Risque                           | Constat sur l'environnement               | Mesure corrective                                                  | Statut   |
|----------------------------------|-------------------------------------------|--------------------------------------------------------------------|----------|
| Absence de supervision           | Aucune alerte au départ                   | Alerte CPU créée (atelier 8)                                       | Corrigé  |
| Actions destructives non tracées | Scripts d'arrêt potentiellement dangereux | <code>vm-power.sh</code> : confirmation, anti-prod, journalisation | Conforme |
| Tags d'ownership manquants       | 12 ressources sans Application            | Normalisation des tags en cours (atelier 3)                        | Partiel  |

## Contrôle des NSG

La règle entrante `Allow-HTTP` (port 80 depuis Internet) est nécessaire pour exposer l'application via le Load Balancer ; elle est acceptable au niveau réseau et se complète d'un WAF en production. La règle `Allow-SSH-Admin` (port 22) est restreinte à une IP `/32`, ce qui réduit fortement la surface de brute-force. Le NSG `nsg-shopeasy-dev-data` applique un `Deny-All-Inbound` et n'autorise que SQL (1433) depuis le subnet web : le subnet data n'est donc pas joignable depuis Internet.

## Synthèse sécurité

Les corrections minimales attendues sont en place : restriction SSH, accès public stockage désactivé, authentification Entra ID, alerte active, journalisation des scripts et absence d'action destructive sans confirmation. Le point ouvert restant est la **complétude des tags d'ownership**, traité côté FinOps/gouvernance.

# Rapport d'exploitation ShopEasy

---

Document de synthèse produit par l'équipe CloudOps — TP3 Administration Azure.  
Date : 26/06/2026 — Périmètre : `rg-shopeasy-dev` (souscription Azure for Students, région `swedencentral` ).

## 1. Contexte et périmètre

---

ShopEasy a déployé son environnement de développement sur Azure (architecture conçue au TP1, industrialisée avec Terraform au TP2). Ce rapport documente l'exploitation quotidienne de cet environnement : inventaire, gouvernance par tags, administration des VM, stockage opérationnel, supervision, sécurité et maîtrise des coûts. Le périmètre est un unique groupe de ressources, `rg-shopeasy-dev`, contenant 14 ressources.

## 2. Inventaire des ressources

---

Inventaire produit par `az resource list` et par [inventory.sh](#) (exports JSON/TSV horodatés) et par [inventory.py](#) (CSV). Synthèse : 2 VM, 2 disques OS, 1 VNet, 2 NSG, 1 Load Balancer, 3 IP publiques, 2 cartes réseau, 1 compte de stockage. Le détail commenté figure dans [01\\_compte\\_rendu\\_ateliers.md](#) (atelier 2).

## 3. Tags et gouvernance

---

Les tags d'exploitation (`Application`, `Environment`, `Owner`, `CostCenter`, `ManagedBy`) ont été appliqués au groupe de ressources et aux deux VM, en mode `Merge` pour conserver les tags posés par Terraform. 12 ressources réseau/stockage restent à taguer avec `Application` : c'est l'action de gouvernance prioritaire.

## 4. État des VM et actions réalisées

---

Deux VM `Standard_B2ts_v2` (`vm-shopeasy-dev-web-1/2`), serveurs web Nginx derrière le Load Balancer. Actions validées : `run-command` distant (Nginx actif), `restart`, et cycle complet `deallocate` → `status` → `start` via `vm-power.sh`. L'application répond en round-robin sur `http://20.91.233.129`.

## 5. Stockage opérationnel

---

Compte `shopeasydevdocsa0rnay`, conteneur privé `operations` créé pour les rapports. Accès public blob **désactivé** au niveau du compte. Dépôt des exports via authentification Entra ID (rôle `Storage Blob Data Contributor`), sans clé de compte.

## 6. Monitoring et alertes

---

Métriques CPU lues via Azure Monitor (CPU moyen < 1 % au repos). Alerte `alert-cpu-high-vm-shopeasy-dev-web-1` créée (CPU moyen > 80 % sur 5 min, sévérité 3). Deux alertes complémentaires recommandées : disponibilité HTTP du Load Balancer et mémoire disponible.

## 7. Scripts produits

---

| Script                      | Rôle                                                |
|-----------------------------|-----------------------------------------------------|
| <code>variables.sh</code>   | Variables d'environnement du TP                     |
| <code>inventory.sh</code>   | Inventaire + compteurs + exports horodatés          |
| <code>vm-power.sh</code>    | Start/stop/deallocate/status sécurisé et journalisé |
| <code>healthcheck.sh</code> | 8 contrôles de santé de l'environnement             |
| <code>inventory.py</code>   | Inventaire via SDK Azure + export CSV               |

## 8. Analyse sécurité

---

SSH restreint à une IP /32 , stockage privé, authentification Entra ID, NSG `Deny-All` sur le subnet data, alerte active et scripts non destructifs sans confirmation. Détail dans [03\\_analyse\\_finops\\_securite.md](#).

## 9. Analyse FinOps

---

Principal levier : désallocation nocturne des VM dev (économie compute ~60-70 %). Compléter les tags, définir un budget mensuel (30-50 €) avec alerte à 80 %. Détail dans [03\\_analyse\\_finops\\_securite.md](#).

## 10. Recommandations d'amélioration

---

1. Compléter le tag `Application` sur les 12 ressources non taguées (gouvernance et FinOps).
  2. Planifier la désallocation automatique des VM dev (cron / Automation runbook appelant `vm-power.sh`).
  3. Ajouter les alertes disponibilité HTTP et mémoire, et créer un budget Azure avec alertes.
  4. En production : remplacer l'accès SSH direct par Azure Bastion et ajouter un WAF devant le Load Balancer.
-

## Tableau de synthèse

| Domaine    | Statut   | Commentaire                                             |
|------------|----------|---------------------------------------------------------|
| Inventaire | Conforme | Exports JSON/TSV/CSV rejouables, tableau complet        |
| Tags       | Partiel  | RG et VM tagués ; 12 ressources sans Application        |
| VM         | Conforme | États maîtrisés, actions start/stop/deallocate validées |
| Stockage   | Conforme | Conteneur privé, accès public désactivé, auth Entra ID  |
| Monitoring | Conforme | Métriques lues, alerte CPU active (seuil justifié)      |
| Sécurité   | Conforme | SSH restreint, NSG maîtrisés, scripts non destructifs   |
| FinOps     | Partiel  | Stratégie d'arrêt définie ; budget et tags à finaliser  |

## TP3 — Quiz final (réponses)

---

- 1. Différence entre `az vm stop` et `az vm deallocate` ?** `az vm stop` éteint le système d'exploitation mais la VM reste **allouée** sur un hôte : le coût compute continue d'être facturé. `az vm deallocate` libère les ressources compute (état Stopped (deallocated)) et **stoppe la facturation compute** ; seuls les disques restent facturés.
- 2. Pourquoi préférer Azure CLI au portail pour une tâche répétitive ?** La CLI est reproductible, scriptable, versionnable et automatisable (cron, CI/CD). Le portail est manuel, lent et sujet aux erreurs humaines, sans trace exploitable.
- 3. À quoi sert l'option `--query` ?** À filtrer et reformater la sortie via JMESPath, pour n'extraire que les champs utiles (ex. nom, état, IP) et produire des sorties directement exploitables dans un script.
- 4. Quel format de sortie pour réutiliser un résultat dans un script Bash ?** `--output tsv` (valeurs séparées par tabulation, sans en-tête ni décoration), facile à parser avec `cut`, `awk` ou une boucle `for`.
- 5. Pourquoi les tags sont-ils importants pour le FinOps ?** Ils permettent de ventiler les coûts par application, environnement ou centre de coût, d'identifier les ressources arrêtables et de refacturer. Sans tags, l'analyse de coût est impossible automatiquement.
- 6. Qu'est-ce qu'un runbook d'exploitation ?** Une procédure documentée et reproductible décrivant comment réaliser une action d'exploitation (démarrer/arrêter un service, vérifier la santé, restaurer, traiter une alerte).
- 7. Pourquoi éviter d'ouvrir SSH à tout Internet ?** Exposer le port 22 à `0.0.0.0/0` offre une surface massive aux attaques par force brute et aux scans automatisés. On restreint SSH à une IP `/32` ou on passe par Azure Bastion / un VPN.
- 8. Quelle commande liste les ressources d'un groupe ?** `az resource list --resource-group <rg> --output table`.
- 9. Quel service Azure suit les métriques et crée des alertes ?** Azure Monitor (métriques, logs, alertes, action groups).
- 10. Différence entre métrique et log ?** Une **métrique** est une valeur numérique échantillonnée dans le temps (CPU %, mémoire) ; un **log** est un enregistrement d'événement horodaté, souvent textuel et détaillé (erreur applicative, accès). Les métriques servent au suivi de tendance et aux alertes seuil ; les logs au diagnostic.
- 11. Pourquoi versionner les scripts d'exploitation ?** Pour tracer l'historique des modifications, permettre la revue de code, revenir en arrière, partager une source unique fiable et éviter les divergences entre membres de l'équipe.
- 12. Rôle de `DefaultAzureCredential` en Python ?** Il fournit une chaîne d'authentification automatique (variables d'environnement, identité managée, session `az login`, etc.) pour s'authentifier auprès d'Azure sans coder de secret en dur.
- 13. Pourquoi tester un script sur un environnement non productif ?** Pour éviter qu'un bug ou une action destructive (arrêt, suppression) n'impacte la production. On valide le comportement sur `dev` avant tout usage réel.

**14. Deux risques d'un script d'administration mal sécurisé ?** (1) Exécution d'une action destructive de masse sans confirmation (arrêt/suppression de toutes les VM) ; (2) fuite de secrets (clés, mots de passe) écrits en clair ou journalisés.

**15. Quelle information doit figurer dans un rapport d'exploitation ?** Le périmètre, l'inventaire, l'état des ressources, les actions réalisées, la supervision/alertes, l'analyse sécurité et FinOps, et des recommandations exploitables.

**16. Pourquoi un budget Azure ne remplace-t-il pas une gouvernance des ressources ?** Un budget alerte sur la dépense mais ne **bloque pas** la création de ressources et ne garantit ni le nettoyage ni le tagging. La gouvernance (tags, Azure Policy, RBAC, revues) traite la cause ; le budget ne traite que le symptôme.

**17. Deux actions simples pour réduire les coûts d'un environnement de dev ?** (1) Désallouer les VM hors heures ouvrées ; (2) supprimer les ressources orphelines (disques, IP publiques non attachées).

**18. Pourquoi journaliser les actions d'un script ?** Pour l'audit (qui a fait quoi et quand), le diagnostic en cas d'incident et la traçabilité réglementaire. Le journal de `vm-power.sh` en est un exemple.

**19. Différence entre IaC et script d'exploitation ?** L'IaC (Terraform) décrit l'**état cible** de l'infrastructure de manière déclarative et idempotente (création/évolution). Un script d'exploitation réalise des **actions ponctuelles ou récurrentes** (impératif) sur une infra existante (arrêt, inventaire, contrôle).

**20. Trois contrôles intégrés dans un `healthcheck.sh` ?** (1) Existence du groupe de ressources et d'au moins une VM ; (2) présence des tags obligatoires ( `Application` , `Owner` ) ; (3) existence d'alertes Monitor, du compte de stockage / conteneur, et d'une règle NSG autorisant le trafic web attendu.

# Annexe A — Arborescence du projet

---

```
cloud/tp3/
├─ variables.sh          (variables d'exploitation - livrable)
├─ scripts/
│   ├─ inventory.sh      (inventaire + compteurs + exports)
│   ├─ vm-power.sh       (start/stop/deallocate/status securise)
│   └─ healthcheck.sh    (8 controles de sante)
├─ python/
│   ├─ inventory.py       (inventaire via SDK Azure + CSV)
│   └─ requirements.txt
├─ exports/              (resources.json/.tsv, inventory.csv, vms-*.txt)
├─ logs/
│   └─ vm-power.log       (journal des actions VM)
├─ screenshots/          (preuves d'execution - Annexe C)
└─ livrables/
    ├─ 01_compte_rendu_ateliers.md
    ├─ 02_quiz_reponses.md
    ├─ 03_analyse_finops_securite.md
    ├─ 04_rapport_exploitation.md
    ├─ 05_note_technique.md
    └─ 06_captures_a_faire.md
```



## Annexe B — Code source

---

Code intégral du mini-kit d'exploitation `tp3/` : variables, scripts Bash et script Python. Aucun secret n'est codé en dur (la souscription et le groupe de ressources sont des identifiants non sensibles, l'authentification passe par la session `az login`).

### variables.sh

---

```
#!/usr/bin/env bash
#
# variables.sh - Variables d'exploitation du TP3 ShopEasy (livrable obligatoire).
#
# Usage : se sourcer dans le shell avant d'executer les commandes/scripts du TP.
#   source tp3/variables.sh
#
# Les valeurs ci-dessous sont issues du déploiement Terraform du TP2
# (terraform output, le 26/06/2026). Elles remplacent les valeurs generiques
# du sujet, adaptees a l'environnement reel "Azure for Students".

# --- Perimetre Azure -----
export RG="rg-shopeasy-dev"
# Region : francecentral est interdite par la policy Azure for Students,
# l'infrastructure est donc deployee a swedencentral (cf. TP1 et TP2).
export LOCATION="swedencentral"

# --- Machines virtuelles (nommage Terraform : vm-<projet>-<env>-web-N) -----
export VM1="vm-shopeasy-dev-web-1"
export VM2="vm-shopeasy-dev-web-2"

# --- Stockage (nom genere avec suffixe aleatoire par Terraform) -----
# Recuperable a tout moment via :
#   terraform -chdir=tp2/terraform output -raw storage_account_name
export STORAGE="shopeasydevdocsa0rnay"
# Conteneur d'exploitation a creer au TP3 (le TP2 fournit deja "documents").
export CONTAINER="operations"

# --- Souscription (necessaire au script Python via le SDK Azure) -----
export AZURE_SUBSCRIPTION_ID="cdca6d99-645a-4251-85e1-f078d1bd66ff"
export AZURE_RESOURCE_GROUP="$RG"

# --- Points d'entree reseau (pour reference / tests applicatifs) -----
export LB_PUBLIC_IP="20.91.233.129"
export VM1_PUBLIC_IP="135.225.37.122"
export VM2_PUBLIC_IP="4.223.122.111"
```

```
echo "Variables TP3 chargees : RG=$RG LOCATION=$LOCATION VM1=$VM1 STORAGE=$STORAGE"
```

## scripts/inventory.sh

```
#!/usr/bin/env bash
#
# inventory.sh - Inventaire d'exploitation d'un groupe de ressources Azure (ShopEasy).
#
# Produit un inventaire rejouable des ressources et des VM, exporte les
# resultats horodates dans exports/ et affiche des compteurs de synthese
# (total, VM, comptes de stockage, ressources sans tag Application) ainsi
# qu'un avertissement si une VM est encore en cours d'execution.
#
# Usage :
#   ./inventory.sh [resource-group]
#   (a default, RG vaut $RG si exporte, sinon rg-shopeasy-dev)

set -euo pipefail

RG="${1:-${RG:-rg-shopeasy-dev}}"
DATE="$(date +%Y%m%d-%H%M%S)"
OUT_DIR="exports"
mkdir -p "$OUT_DIR"

echo "Inventaire Azure - groupe de ressources : $RG"
echo "Date : $DATE"
echo "-----"

# Verifie que le groupe existe avant tout traitement.
if ! az group show --name "$RG" >/dev/null 2>&1; then
    echo "Erreur : groupe de ressources '$RG' introuvable." >&2
    exit 1
fi

echo "Export des ressources (JSON)..."
az resource list \
    --resource-group "$RG" \
    --query "[].{name:name,type:type,location:location,id:id,tags:tags}" \
    --output json > "$OUT_DIR/resources-$DATE.json"

echo "Export des VM (table)..."
az vm list \
    --resource-group "$RG" \
```

```

--show-details \
--query "[].{name:name,powerState:powerState,publicIps:publicIps,vmSize:hardwareProfile.vmSize}" \
--output table > "$OUT_DIR/vms-$DATE.txt"

# --- Compteurs de synthese -----
TOTAL=$(az resource list --resource-group "$RG" --query "length(@)" --output tsv)
NB_VM=$(az resource list --resource-group "$RG" \
--query "length([?type=='Microsoft.Compute/virtualMachines'])" --output tsv)
NB_STORAGE=$(az resource list --resource-group "$RG" \
--query "length([?type=='Microsoft.Storage/storageAccounts'])" --output tsv)
NB_UNTAGGED=$(az resource list --resource-group "$RG" \
--query "length([?tags.Application==null])" --output tsv)

echo "-----"
echo "Synthese :"
echo "  Ressources totales      : $TOTAL"
echo "  Machines virtuelles     : $NB_VM"
echo "  Comptes de stockage      : $NB_STORAGE"
echo "  Ressources sans tag App. : $NB_UNTAGGED"

# Avertissement FinOps si une VM tourne encore (cout compute en cours).
RUNNING=$(az vm list --resource-group "$RG" --show-details \
--query "[?powerState=='VM running'].name" --output tsv || true)
if [[ -n "$RUNNING" ]]; then
  echo "-----"
  echo "ATTENTION : VM en cours d'execution (cout compute facture) :"
  while IFS= read -r vm; do
    [[ -n "$vm" ]] && echo "  - $vm"
  done <<< "$RUNNING"
fi

echo "-----"
echo "Export termine dans $OUT_DIR/ :"
echo "  - resources-$DATE.json"
echo "  - vms-$DATE.txt"

```

## scripts/vm-power.sh

```

#!/usr/bin/env bash
#
# vm-power.sh - Pilotage de l'alimentation des VM d'un groupe de ressources.
#
# Actions : start | stop | deallocate | status sur toutes les VM du groupe.
#

```

```

# Mesures de securite :
# - refus d'executer sur un groupe dont le nom contient "prod" ;
# - confirmation interactive avant toute action destructive (stop/deallocate) ;
# - journalisation de chaque action dans logs/vm-power.log.
#
# Usage :
# ./vm-power.sh <resource-group> <start|stop|deallocate|status>

set -euo pipefail

RG="${1:-}"
ACTION="${2:-}"
LOG_DIR="logs"
LOG_FILE="$LOG_DIR/vm-power.log"
mkdir -p "$LOG_DIR"

# Journalise un message horodate, a la fois a l'ecran et dans le fichier de log.
log() {
    local msg="$1"
    echo "$(date '+%Y-%m-%d %H:%M:%S') | $RG | $ACTION | $msg" >> "$LOG_FILE"
    echo "$msg"
}

if [[ -z "$RG" || -z "$ACTION" ]]; then
    echo "Usage : $0 <resource-group> <start|stop|deallocate|status>" >&2
    exit 1
fi

# Garde-fou : jamais d'action de masse sur un environnement de production.
if [[ "$RG" == *prod* ]]; then
    echo "Refus : '$RG' ressemble a un groupe de PRODUCTION. Action interdite par ce script." >&2
    exit 2
fi

# Confirmation explicite avant les actions destructives.
confirm() {
    local prompt="$1"
    read -r -p "$prompt [oui/non] " reply
    [[ "$reply" == "oui" ]]
}

VMS=$(az vm list --resource-group "$RG" --query "[].name" --output tsv)
if [[ -z "$VMS" ]]; then
    log "Aucune VM trouvee dans $RG"
    exit 0
fi

```

```

if [[ "$ACTION" == "stop" || "$ACTION" == "deallocate" ]]; then
    echo "VM concernees par '$ACTION' :"
    echo "$VMS" | sed 's/^/ - /'
    if ! confirm "Confirmer l'action '$ACTION' sur ces VM ?"; then
        log "Action '$ACTION' annulee par l'utilisateur"
        exit 0
    fi
fi

for VM in $VMS; do
    log "Traitement de la VM : $VM"
    case "$ACTION" in
        start)
            az vm start --resource-group "$RG" --name "$VM" >/dev/null
            log "VM demarree : $VM"
            ;;
        stop)
            az vm stop --resource-group "$RG" --name "$VM" >/dev/null
            log "VM arrete (OS) : $VM"
            ;;
        deallocate)
            az vm deallocate --resource-group "$RG" --name "$VM" >/dev/null
            log "VM desallouee (cout compute stoppe) : $VM"
            ;;
        status)
            STATE=$(az vm get-instance-view \
                --resource-group "$RG" \
                --name "$VM" \
                --query "instanceView.statuses[?starts_with(code, 'PowerState/')].displayStatus" \
                --output tsv)
            log "Etat : $VM -> $STATE"
            ;;
        *)
            echo "Action inconnue : $ACTION" >&2
            exit 1
            ;;
    esac
done

```

## scripts/healthcheck.sh

```

#!/usr/bin/env bash
#

```

```

# healthcheck.sh - Controle de sante d'exploitation de l'environnement ShopEasy.
#
# Verifie qu'un groupe de ressources Azure est dans un etat acceptable :
# 1. existence du groupe de ressources ;
# 2. presence d'au moins une VM et affichage de leur etat ;
# 3. ressources sans tag Application ;
# 4. presence d'un tag Owner sur le groupe ;
# 5. alertes Azure Monitor configurees ;
# 6. existence du compte de stockage ;
# 7. existence du conteneur "operations" ;
# 8. au moins une regle NSG autorisant HTTP ou HTTPS.
#
# Code de sortie : 0 si tout est OK, 1 si au moins un avertissement.
#
# Usage :
# ./healthcheck.sh [resource-group] [storage-account] [container]

set -euo pipefail

RG="${1:-${RG:-rg-shopeasy-dev}}"
STORAGE="${2:-${STORAGE:-}}"
CONTAINER="${3:-${CONTAINER:-operations}}"
STATUS=0

echo "Controle de sante Azure - $RG"
echo "===== "

# 1. Groupe de ressources -----
echo "1. Verification du groupe de ressources"
if az group show --name "$RG" >/dev/null 2>&1; then
    echo "    OK - groupe de ressources trouve"
else
    echo "    KO - groupe de ressources introuvable"
    exit 1
fi

# 2. Machines virtuelles -----
echo "2. Verification des VM"
NB_VM=$(az vm list --resource-group "$RG" --query "length(@)" --output tsv)
if [[ "$NB_VM" -ge 1 ]]; then
    echo "    OK - $NB_VM VM presente(s)"
    az vm list --resource-group "$RG" --show-details \
        --query "[].{name:name,state:powerState,ip:publicIps}" --output table | sed 's/^/      /'
else
    echo "    WARN- aucune VM dans le groupe"
    STATUS=1

```

```

fi

# 3. Tag Application sur les ressources -----
echo "3. Verification des ressources sans tag Application"
UNTAGGED=$(az resource list --resource-group "$RG" \
  --query "length([?tags.Application==null])" --output tsv)
if [[ "$UNTAGGED" -gt 0 ]]; then
  echo "  WARN- $UNTAGGED ressource(s) sans tag Application"
  STATUS=1
else
  echo "  OK - tag Application present partout"
fi

# 4. Tag Owner sur le groupe de ressources -----
echo "4. Verification du tag Owner sur le groupe"
OWNER=$(az group show --name "$RG" --query "tags.Owner || tags.owner" --output tsv)
if [[ -n "$OWNER" && "$OWNER" != "None" ]]; then
  echo "  OK - Owner = $OWNER"
else
  echo "  WARN- aucun tag Owner sur le groupe"
  STATUS=1
fi

# 5. Alertes Azure Monitor -----
echo "5. Verification des alertes Azure Monitor"
ALERTS=$(az monitor metrics alert list --resource-group "$RG" --query "length(@)" --output tsv)
echo "  Nombre d'alertes : $ALERTS"
if [[ "$ALERTS" -eq 0 ]]; then
  echo "  WARN- aucune alerte configuree"
  STATUS=1
else
  echo "  OK - alerte(s) presente(s)"
fi

# 6. Compte de stockage -----
echo "6. Verification du compte de stockage"
if [[ -z "$STORAGE" ]]; then
  STORAGE=$(az storage account list --resource-group "$RG" --query "[0].name" --output tsv)
fi
if [[ -n "$STORAGE" ]] && az storage account show --name "$STORAGE" --resource-group "$RG" >/dev/null
2>&1; then
  echo "  OK - compte de stockage $STORAGE present"
else
  echo "  WARN- compte de stockage absent"
  STATUS=1
fi

```

```

# 7. Conteneur operations -----
echo "7. Verification du conteneur '$CONTAINER'"
if [[ -n "$STORAGE" ]] && az storage container show \
    --account-name "$STORAGE" --name "$CONTAINER" --auth-mode login >/dev/null 2>&1; then
    echo "    OK - conteneur '$CONTAINER' present"
else
    echo "    WARN- conteneur '$CONTAINER' absent (ou droits insuffisants)"
    STATUS=1
fi

# 8. Regle NSG HTTP/HTTPS -----
echo "8. Verification d'une regle NSG autorisant HTTP/HTTPS"
HTTP_RULES=0
for NSG in $(az network nsg list --resource-group "$RG" --query "[].name" --output tsv); do
    N=$(az network nsg rule list --resource-group "$RG" --nsg-name "$NSG" \
        --query "length([?access=='Allow' && direction=='Inbound' && (destinationPortRange=='80' ||
destinationPortRange=='443']]))" \
        --output tsv)
    HTTP_RULES=$((HTTP_RULES + N))
done
if [[ "$HTTP_RULES" -ge 1 ]]; then
    echo "    OK - $HTTP_RULES regle(s) NSG autorisant HTTP/HTTPS"
else
    echo "    WARN- aucune regle NSG n'autorise HTTP/HTTPS"
    STATUS=1
fi

echo "====="
if [[ "$STATUS" -eq 0 ]]; then
    echo "Resultat : environnement SAIN (tous les controles OK)."
```

```

else
    echo "Resultat : avertissement(s) detecte(s) - voir lignes WARN ci-dessus."
fi
exit $STATUS
```

## python/inventory.py

```

#!/usr/bin/env python3

"""Inventaire programmable des ressources Azure d'un groupe de ressources, via le SDK Azure.

S'authentifie avec DefaultAzureCredential (reutilise la session `az login`),
liste les ressources et les VM du groupe, affiche un resume a l'ecran et
produit un fichier CSV exploitable dans Excel / LibreOffice Calc.
```



Colonnes du CSV : nom, type, region, tags, rôle suppose dans l'architecture.

Variables d'environnement attendues :

```
AZURE_SUBSCRIPTION_ID  (obligatoire)
AZURE_RESOURCE_GROUP   (default : rg-shopeasy-dev)
```

Usage :

```
pip install -r requirements.txt
export AZURE_SUBSCRIPTION_ID=<subscription-id>
export AZURE_RESOURCE_GROUP="rg-shopeasy-dev"
python inventory.py [chemin_csv]
```

```
"""
```

```
from __future__ import annotations
```

```
import csv
```

```
import os
```

```
import sys
```

```
from azure.identity import DefaultAzureCredential
```

```
from azure.mgmt.compute import ComputeManagementClient
```

```
# Selon la version du SDK, ResourceManagementClient est expose soit au niveau
```

```
# du package azure.mgmt.resource, soit dans le sous-module .resources.
```

```
try:
```

```
    from azure.mgmt.resource import ResourceManagementClient
```

```
except ImportError:
```

```
    from azure.mgmt.resource.resources import ResourceManagementClient
```

```
# Correspondance type Azure -> rôle fonctionnel dans l'architecture ShopEasy.
```

```
ROLE_BY_TYPE = {
```

```
    "Microsoft.Compute/virtualMachines": "Serveur web (VM Linux Nginx)",
```

```
    "Microsoft.Compute/disks": "Disque OS managé d'une VM",
```

```
    "Microsoft.Network/virtualNetworks": "Réseau privé (VNet)",
```

```
    "Microsoft.Network/networkSecurityGroups": "Pare-feu réseau (NSG)",
```

```
    "Microsoft.Network/loadBalancers": "Répartiteur de charge HTTP",
```

```
    "Microsoft.Network/publicIPAddresses": "Adresse IP publique",
```

```
    "Microsoft.Network/networkInterfaces": "Carte réseau d'une VM",
```

```
    "Microsoft.Storage/storageAccounts": "Compte de stockage (documents/rapports)",
```

```
}
```

```
def guess_role(resource_type: str) -> str:
```

```
    """Renvoie le rôle suppose pour un type de ressource, ou une valeur générique."""
```

```
    return ROLE_BY_TYPE.get(resource_type, "Autre / non classé")
```

```

def format_tags(tags: dict | None) -> str:
    """Serialise les tags en chaine 'cle=valeur;...' lisible dans une cellule CSV."""
    if not tags:
        return ""
    return ";".join(f"{k}={v}" for k, v in sorted(tags.items()))

def main() -> None:
    subscription_id = os.environ.get("AZURE_SUBSCRIPTION_ID")
    resource_group = os.environ.get("AZURE_RESOURCE_GROUP", "rg-shopeasy-dev")
    csv_path = sys.argv[1] if len(sys.argv) > 1 else "exports/inventory.csv"

    if not subscription_id:
        raise SystemExit("Variable AZURE_SUBSCRIPTION_ID manquante.")

    credential = DefaultAzureCredential()
    resource_client = ResourceManagementClient(credential, subscription_id)
    compute_client = ComputeManagementClient(credential, subscription_id)

    print(f"Inventaire du groupe : {resource_group}")
    print("Ressources :")

    rows: list[dict[str, str]] = []
    for res in resource_client.resources.list_by_resource_group(resource_group):
        print(f"- {res.name} | {res.type} | {res.location}")
        rows.append(
            {
                "nom": res.name,
                "type": res.type,
                "region": res.location,
                "tags": format_tags(res.tags),
                "role": guess_role(res.type),
            }
        )

    print("\nMachines virtuelles :")
    for vm in compute_client.virtual_machines.list(resource_group):
        size = vm.hardware_profile.vm_size
        print(f"- {vm.name} | taille={size} | region={vm.location}")

    # Ecriture du CSV (UTF-8 avec BOM pour une ouverture propre dans Excel).
    os.makedirs(os.path.dirname(csv_path) or ".", exist_ok=True)
    with open(csv_path, "w", newline="", encoding="utf-8-sig") as fh:
        writer = csv.DictWriter(

```

```
        fh, fieldnames=["nom", "type", "region", "tags", "role"], delimiter=";"
    )
    writer.writeheader()
    writer.writerows(rows)

    print(f"\n{len(rows)} ressource(s) exportee(s) dans {csv_path}")

if __name__ == "__main__":
    main()
```

## python/requirements.txt

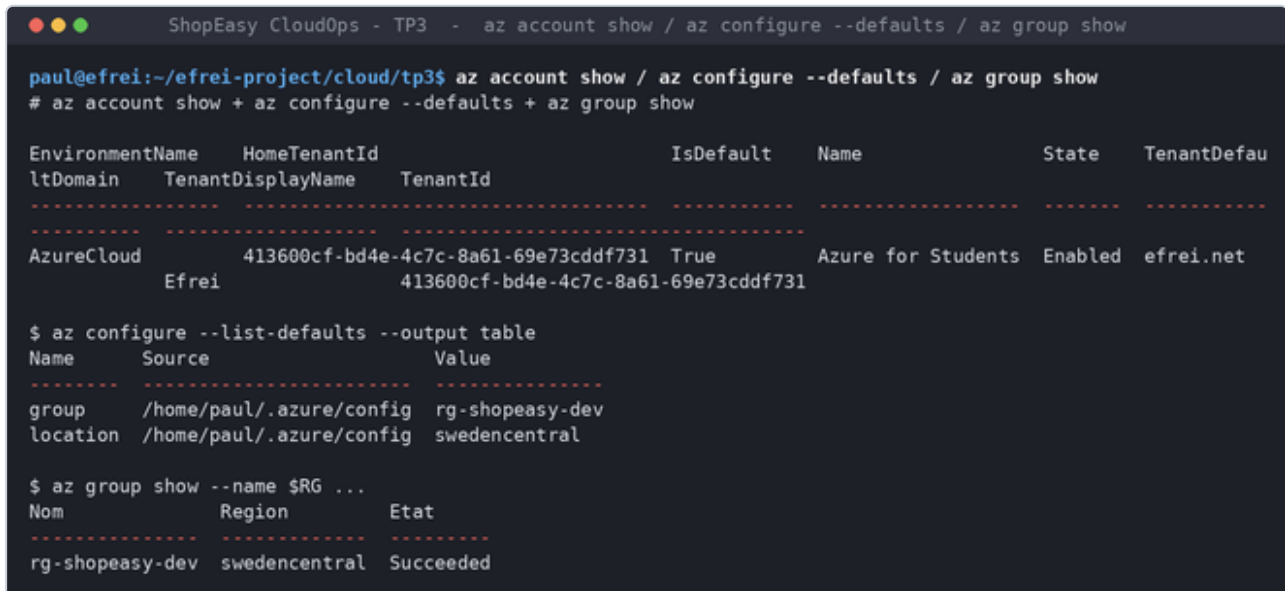
---

```
azure-identity>=1.17
azure-mgmt-resource>=23.0
azure-mgmt-compute>=33.0
```

## Annexe C — Preuves d'exécution

Captures des commandes Azure CLI, des scripts Bash, de la supervision et du script Python, exécutés sur l'environnement réel. La correspondance capture → atelier est détaillée dans `tp3/livrables/06_captures_a_faire.md`.

### atelier\_01-account-defaults.png



```
ShopEasy CloudOps - TP3 - az account show / az configure --defaults / az group show

paul@efrei:~/efrei-project/cloud/tp3$ az account show / az configure --defaults / az group show
# az account show + az configure --defaults + az group show

EnvironmentName  HomeTenantId  IsDefault  Name  State  TenantDefau
ltDomain  TenantDisplayName  TenantId
-----
AzureCloud      413600cf-bd4e-4c7c-8a61-69e73cddf731  True      Azure for Students  Enabled  efrei.net
Efrei           413600cf-bd4e-4c7c-8a61-69e73cddf731

$ az configure --list-defaults --output table
Name      Source  Value
-----
group     /home/paul/.azure/config  rg-shopeasy-dev
location  /home/paul/.azure/config  swedencentral

$ az group show --name $RG ...
Nom      Region  Etat
-----
rg-shopeasy-dev  swedencentral  Succeeded
```

## atelier\_02-resource-list.png

```
ShopEasy CloudOps - TP3 - az resource list --query [].{Nom,Type,Region}

paul@efrei:~/efrei-project/cloud/tp3$ az resource list --query [].{Nom,Type,Region}
Nom                                     Type                                     Regi
on
-----
pip-shopeasy-dev-web-1                 Microsoft.Network/publicIPAddresses     swed
encentral
nsg-shopeasy-dev-web                   Microsoft.Network/networkSecurityGroups  swed
encentral
nsg-shopeasy-dev-data                   Microsoft.Network/networkSecurityGroups  swed
encentral
pip-shopeasy-dev-lb                     Microsoft.Network/publicIPAddresses     swed
encentral
vnet-shopeasy-dev                       Microsoft.Network/virtualNetworks       swed
encentral
pip-shopeasy-dev-web-2                 Microsoft.Network/publicIPAddresses     swed
encentral
shopeasydevdocsa0rnay                  Microsoft.Storage/storageAccounts       swed
encentral
lb-shopeasy-dev-web                     Microsoft.Network/loadBalancers          swed
encentral
nic-shopeasy-dev-web-2                  Microsoft.Network/networkInterfaces      swed
encentral
nic-shopeasy-dev-web-1                  Microsoft.Network/networkInterfaces      swed
encentral
vm-shopeasy-dev-web-1                   Microsoft.Compute/virtualMachines        swed
encentral
vm-shopeasy-dev-web-2                   Microsoft.Compute/virtualMachines        swed
encentral
vm-shopeasy-dev-web-1_0sDisk_1_966ad9888287402d8c178819c33517c4 Microsoft.Compute/disks                  swed
encentral
vm-shopeasy-dev-web-2_0sDisk_1_d26a1dc79f9043d987aff3fe36174320 Microsoft.Compute/disks                  swed
encentral

$ az resource list ... --query "[] .type" -o tsv | sort | uniq -c
  2 Microsoft.Compute/disks
  2 Microsoft.Compute/virtualMachines
  1 Microsoft.Network/loadBalancers
  2 Microsoft.Network/networkInterfaces
  2 Microsoft.Network/networkSecurityGroups
  3 Microsoft.Network/publicIPAddresses
  1 Microsoft.Network/virtualNetworks
  1 Microsoft.Storage/storageAccounts
```

## atelier\_03-tags.png

```
ShopEasy CloudOps - TP3 - az tag update --operation Merge (RG + VM)

paul@efrei:~/efrei-project/cloud/tp3$ az tag update --operation Merge (RG + VM)
# az group show --query tags (apres merge)
{
  "Application": "shopeasy",
  "CostCenter": "cloud-lab",
  "ManagedBy": "cli",
  "cost_center": "cloud-training",
  "environment": "dev",
  "managed_by": "terraform",
  "owner": "formation",
  "project": "shopeasy"
}

# az vm show -n $VM1 --query tags
{
  "Application": "shopeasy",
  "Role": "web",
  "cost_center": "cloud-training",
  "environment": "dev",
  "managed_by": "terraform",
  "owner": "formation",
  "project": "shopeasy"
}
```

## atelier\_04-vm-runcommand.png

```
ShopEasy CloudOps - TP3 - az vm list --show-details / run-command / restart

paul@efrei:~/efrei-project/cloud/tp3$ az vm list --show-details / run-command / restart
# az vm list --show-details

```

| Nom                   | Etat       | IP             | Taille           |
|-----------------------|------------|----------------|------------------|
| vm-shopeasy-dev-web-1 | VM running | 135.225.37.122 | Standard_B2ts_v2 |
| vm-shopeasy-dev-web-2 | VM running | 4.223.122.111  | Standard_B2ts_v2 |

```

# az vm run-command invoke (nginx)
Enable succeeded:
[stdout]
vm-shopeasy-dev-web-1
08:09:56 up 4 min, 0 users, load average: 0.10, 0.15, 0.08
active
ShopEasy - serveur web 1

[stderr]

# az vm restart -> etat:
VM running
```

## atelier\_05-inventory-script.png

```
ShopEasy CloudOps - TP3 - ./scripts/inventory.sh $RG

paul@efrei:~/efrei-project/cloud/tp3$ ./scripts/inventory.sh $RG
Inventaire Azure - groupe de ressources : rg-shopeasy-dev
Date : 20260626-101400
-----
Export des ressources (JSON)...
Export des VM (table)...
-----
Synthese :
  Ressources totales      : 14
  Machines virtuelles    : 2
  Comptes de stockage    : 1
  Ressources sans tag App. : 12
-----
ATTENTION : VM en cours d'execution (cout compute facture) :
- vm-shopeasy-dev-web-1
- vm-shopeasy-dev-web-2
-----
Export termine dans exports/ :
- resources-20260626-101400.json
- vms-20260626-101400.txt
```

## atelier\_06-vm-power.png

```
ShopEasy CloudOps - TP3 - ./scripts/vm-power.sh $RG status|deallocate|start

paul@efrei:~/efrei-project/cloud/tp3$ ./scripts/vm-power.sh $RG status|deallocate|start
# ./vm-power.sh $RG status
Traitement de la VM : vm-shopeasy-dev-web-1
Etat : vm-shopeasy-dev-web-1 -> VM running
Traitement de la VM : vm-shopeasy-dev-web-2
Etat : vm-shopeasy-dev-web-2 -> VM running

# ./vm-power.sh $RG deallocate (confirmation 'oui')
VM concernees par 'deallocate' :
- vm-shopeasy-dev-web-1
- vm-shopeasy-dev-web-2
Traitement de la VM : vm-shopeasy-dev-web-1
VM desallouee (cout compute stoppe) : vm-shopeasy-dev-web-1
Traitement de la VM : vm-shopeasy-dev-web-2
VM desallouee (cout compute stoppe) : vm-shopeasy-dev-web-2

# ./vm-power.sh $RG start
Traitement de la VM : vm-shopeasy-dev-web-1
VM demarree : vm-shopeasy-dev-web-1
Traitement de la VM : vm-shopeasy-dev-web-2
VM demarree : vm-shopeasy-dev-web-2

# logs/vm-power.log
2026-06-26 10:20:37 | rg-shopeasy-dev | status | Traitement de la VM : vm-shopeasy-dev-web-2
2026-06-26 10:20:39 | rg-shopeasy-dev | status | Etat : vm-shopeasy-dev-web-2 -> VM deallocated
2026-06-26 10:20:40 | rg-shopeasy-dev | start | Traitement de la VM : vm-shopeasy-dev-web-1
2026-06-26 10:21:23 | rg-shopeasy-dev | start | VM demarree : vm-shopeasy-dev-web-1
2026-06-26 10:21:23 | rg-shopeasy-dev | start | Traitement de la VM : vm-shopeasy-dev-web-2
2026-06-26 10:21:37 | rg-shopeasy-dev | start | VM demarree : vm-shopeasy-dev-web-2
```

## atelier\_07-storage-blobs.png

```
ShopEasy CloudOps - TP3 - az storage account show / az storage blob list

paul@efrei:~/efrei-project/cloud/tp3$ az storage account show / az storage blob list
# az storage account show (apres durcissement)
Nom          AccesPublicBlob  TLS
-----
shopeasydevdocsa0rnay  False           TLS1_2

# az storage blob list -c operations
Nom          Taille
-----
inventaire/resources-20260626-101400.json  6793
inventaire/resources.tsv                  1059
rapports/rapport-test.txt                 62
```

## atelier\_08-monitor-alert.png

```
ShopEasy CloudOps - TP3 - az monitor metrics list / alert create

paul@efrei:~/efrei-project/cloud/tp3$ az monitor metrics list / alert create
# az monitor metrics list-definitions (extrait)
Metrique          Unite
-----
Percentage CPU    Percent
Disk Read Bytes   Bytes
OS Disk IOPS Consumed Percentage Percent
Network In Total  Bytes
Available Memory Bytes Bytes

# az monitor metrics list --metric 'Percentage CPU'
Heure          CPU_moyen
-----
2026-06-26T08:13:00Z  0.87
2026-06-26T08:14:00Z  0.335
2026-06-26T08:15:00Z  0.125
2026-06-26T08:16:00Z  0.13
2026-06-26T08:17:00Z  0.125

# az monitor metrics alert list
Nom          Severite  Active  Fenetre
-----
alert-cpu-high-vm-shopeasy-dev-web-1  3        True    PT5M
```



## atelier\_09-healthcheck.png

```
ShopEasy CloudOps - TP3 - ./scripts/healthcheck.sh $RG $STORAGE operations

paul@efrei:~/efrei-project/cloud/tp3$ ./scripts/healthcheck.sh $RG $STORAGE operations
Controle de sante Azure - rg-shopeasy-dev
=====
1. Verification du groupe de ressources
   OK - groupe de ressources trouve
2. Verification des VM
   OK - 2 VM presente(s)
      Name                State      Ip
      -----
      vm-shopeasy-dev-web-1 VM running 135.225.37.122
      vm-shopeasy-dev-web-2 VM running 4.223.122.111
3. Verification des ressources sans tag Application
   WARN- 12 ressource(s) sans tag Application
4. Verification du tag Owner sur le groupe
   OK - Owner = formation
5. Verification des alertes Azure Monitor
   Nombre d'alertes : 1
   OK - alerte(s) presente(s)
6. Verification du compte de stockage
   OK - compte de stockage shopeasydevdocsa0rnay present
7. Verification du conteneur 'operations'
   OK - conteneur 'operations' present
8. Verification d'une regle NSG autorisant HTTP/HTTPS
   OK - 1 regle(s) NSG autorisant HTTP/HTTPS
=====
Resultat : avertissement(s) detecte(s) - voir lignes WARN ci-dessus.
```

## atelier\_10-python-csv.png

```
ShopEasy CloudOps - TP3 - python inventory.py exports/inventory.csv

paul@efrei:~/efrei-project/cloud/tp3$ python inventory.py exports/inventory.csv
Inventaire du groupe : rg-shopeasy-dev
Ressources :
- pip-shopeasy-dev-web-1 | Microsoft.Network/publicIPAddresses | swedencentral
- nsg-shopeasy-dev-web | Microsoft.Network/networkSecurityGroups | swedencentral
- nsg-shopeasy-dev-data | Microsoft.Network/networkSecurityGroups | swedencentral
- pip-shopeasy-dev-lb | Microsoft.Network/publicIPAddresses | swedencentral
- vnet-shopeasy-dev | Microsoft.Network/virtualNetworks | swedencentral
- pip-shopeasy-dev-web-2 | Microsoft.Network/publicIPAddresses | swedencentral
- shopeasydevdocsa0rnay | Microsoft.Storage/storageAccounts | swedencentral
- lb-shopeasy-dev-web | Microsoft.Network/loadBalancers | swedencentral
- nic-shopeasy-dev-web-2 | Microsoft.Network/networkInterfaces | swedencentral
- nic-shopeasy-dev-web-1 | Microsoft.Network/networkInterfaces | swedencentral
- vm-shopeasy-dev-web-1 | Microsoft.Compute/virtualMachines | swedencentral
- vm-shopeasy-dev-web-2 | Microsoft.Compute/virtualMachines | swedencentral
- vm-shopeasy-dev-web-1_0sDisk_1_966ad9888287402d8c178819c33517c4 | Microsoft.Compute/disks | swedencentral
- vm-shopeasy-dev-web-2_0sDisk_1_d26a1dc79f9043d987aff3fe36174320 | Microsoft.Compute/disks | swedencentral
- alert-cpu-high-vm-shopeasy-dev-web-1 | Microsoft.Insights/metricalerts | global

Machines virtuelles :
- vm-shopeasy-dev-web-1 | taille=Standard_B2ts_v2 | region=swedencentral
- vm-shopeasy-dev-web-2 | taille=Standard_B2ts_v2 | region=swedencentral

15 ressource(s) exportee(s) dans exports/inventory.csv

# tete de exports/inventory.csv
nom;type;region;tags;role
pip-shopeasy-dev-web-1;Microsoft.Network/publicIPAddresses;swedencentral;"cost_center=cloud-training;environme
nt=dev;managed_by=terraform;owner=formation;project=shopeasy";Adresse IP publique
[14 more lines]
```